POSTGRES ON OPENSHIFT WORKSHOP

Stephen Hall - Senior Sales Engineer, EDB
Soren Boss - Strategic Account Executive, EDB

2th of June, 2026

Agenda

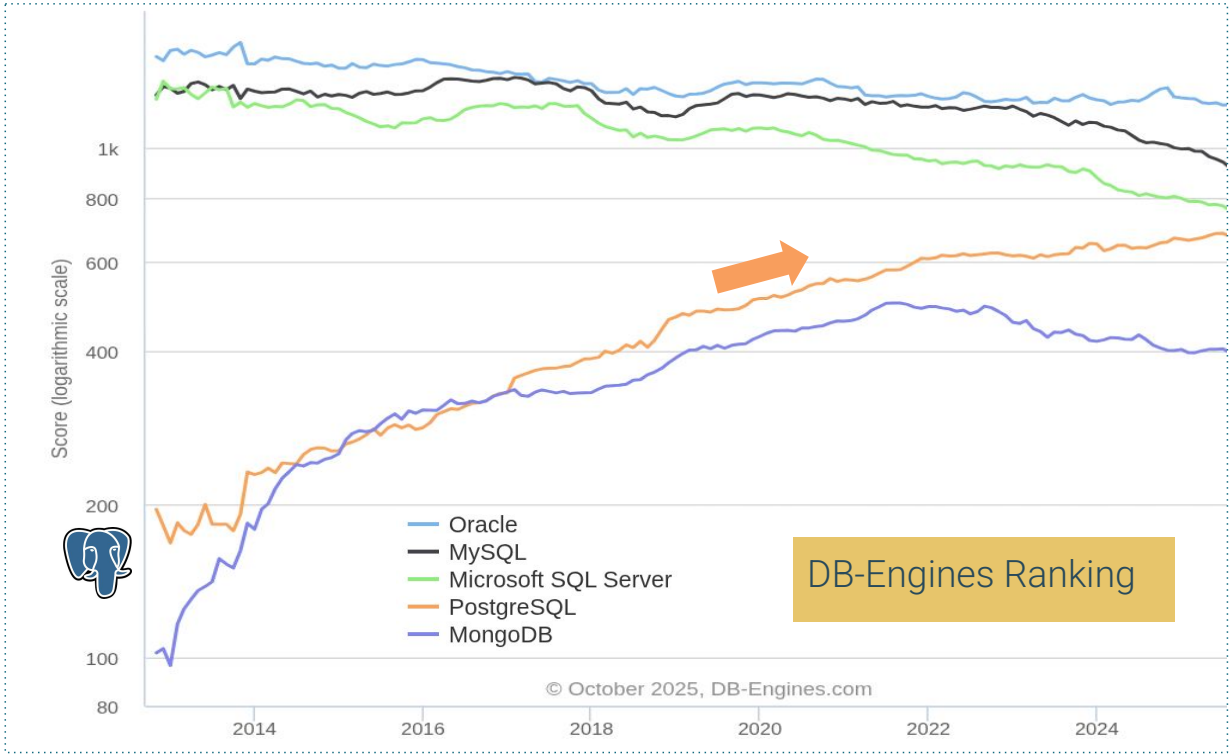
Commencer	Fin	Session
14:00	14:30	Intro to Postgres, EDB, Cloud Native PG operator, Architecture
14:30	15:55	Hands-on workshop!
15:55	16:00	Wrap up and questions!

©EDB 2025— ALL RIGHTS RESERVED.

Postgres? Why we here today?

©EDB 2025 – ALL RIGHTS RESERVED.

PostgreSQL is Booming !

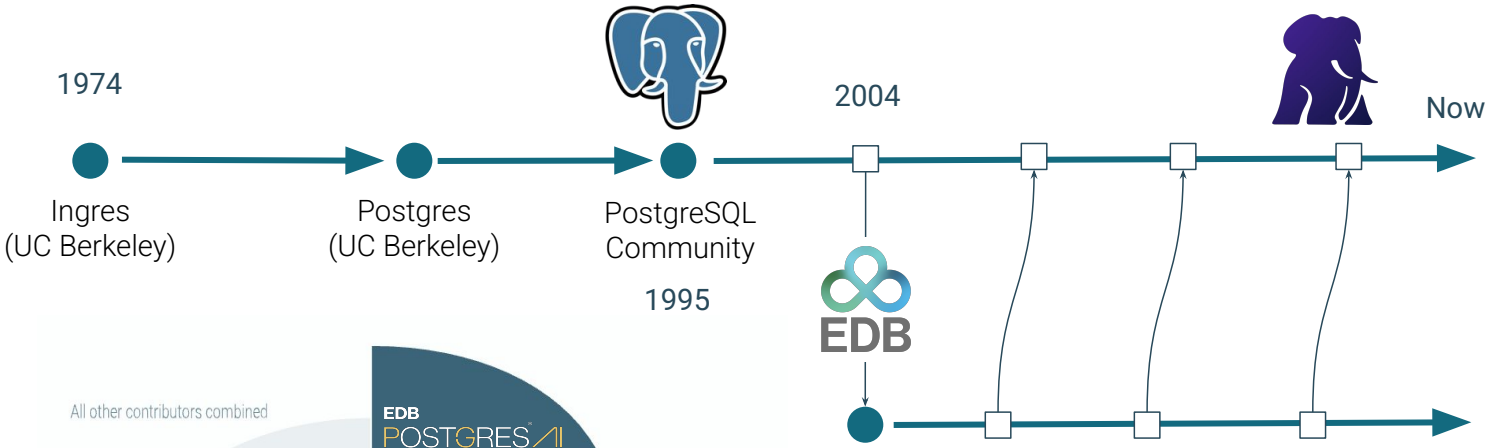


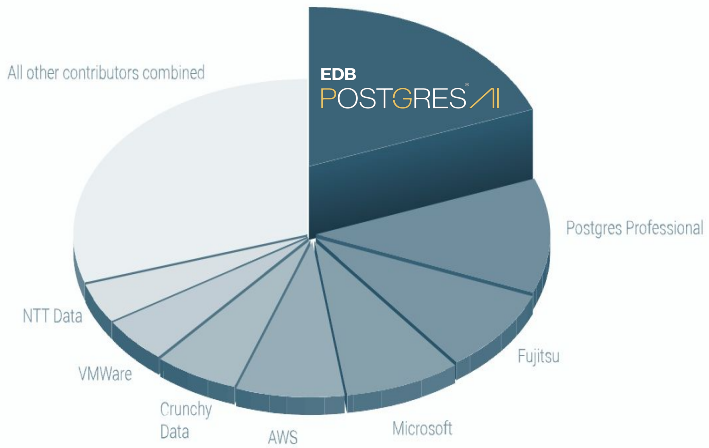
35%

Thirty-five percent of Enterprises will consider Postgres for their next project

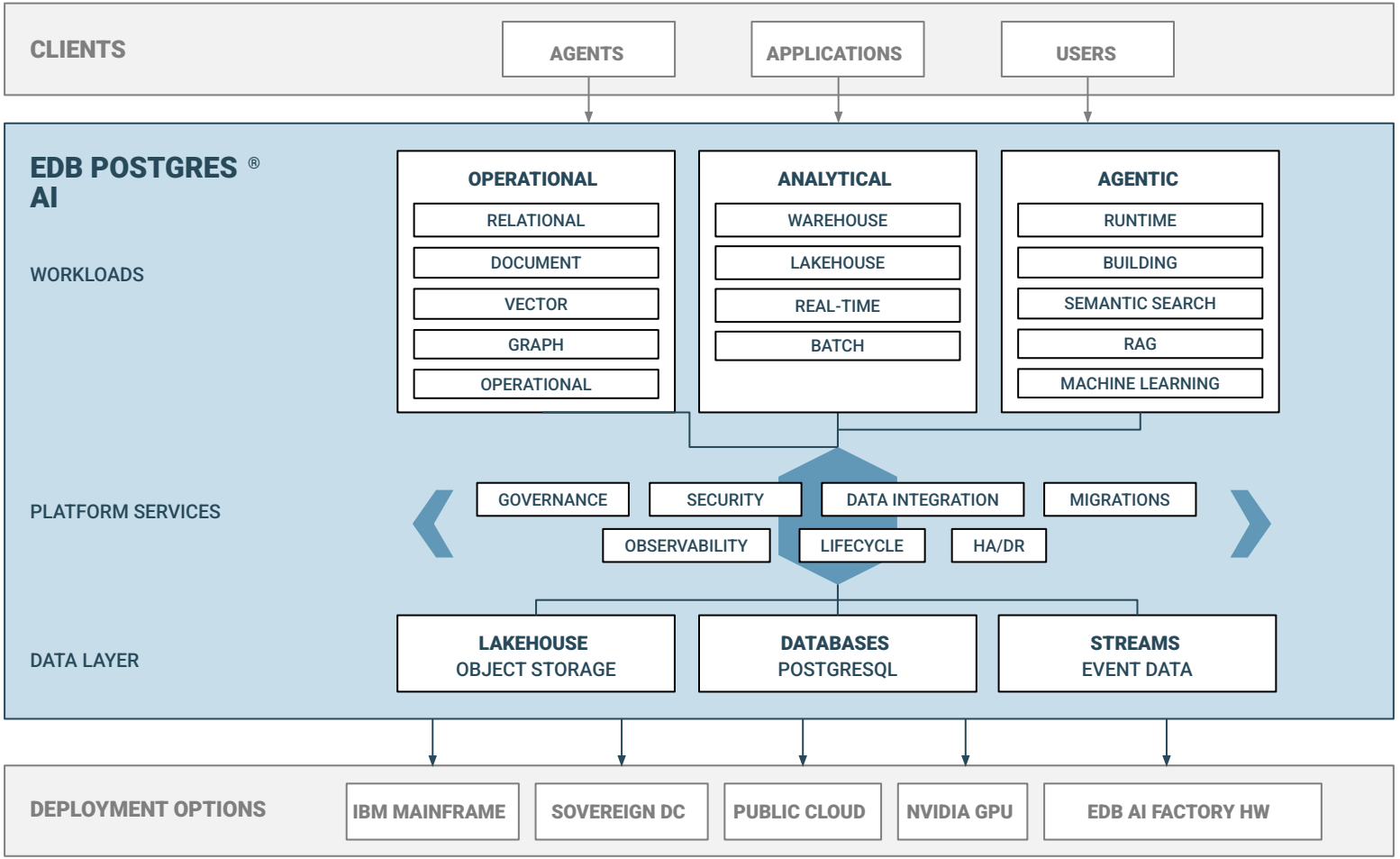
©EDB 2024 — ALL RIGHTS RESERVED.

EnterpriseDB Biggest PostgreSQL Contributor





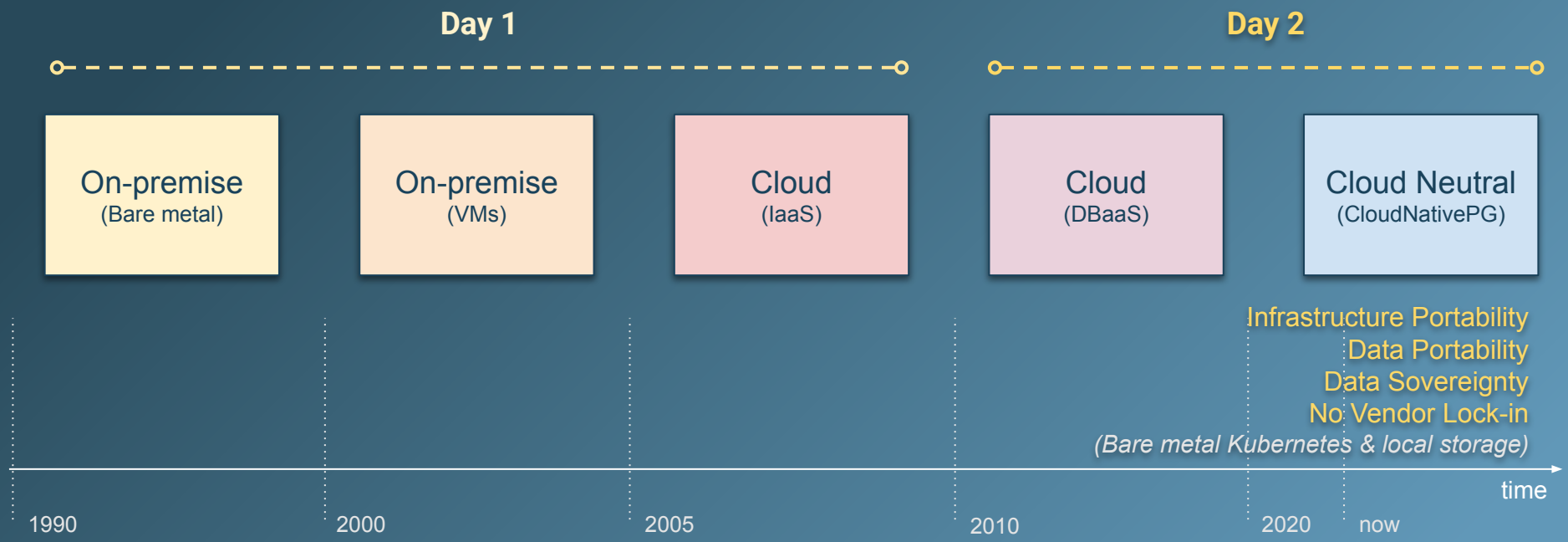
©EDB 2025 — ALL RIGHTS RESERVED.



©EDB 2026 — ALL RIGHTS RESERVED.

The evolution of Postgres use cases

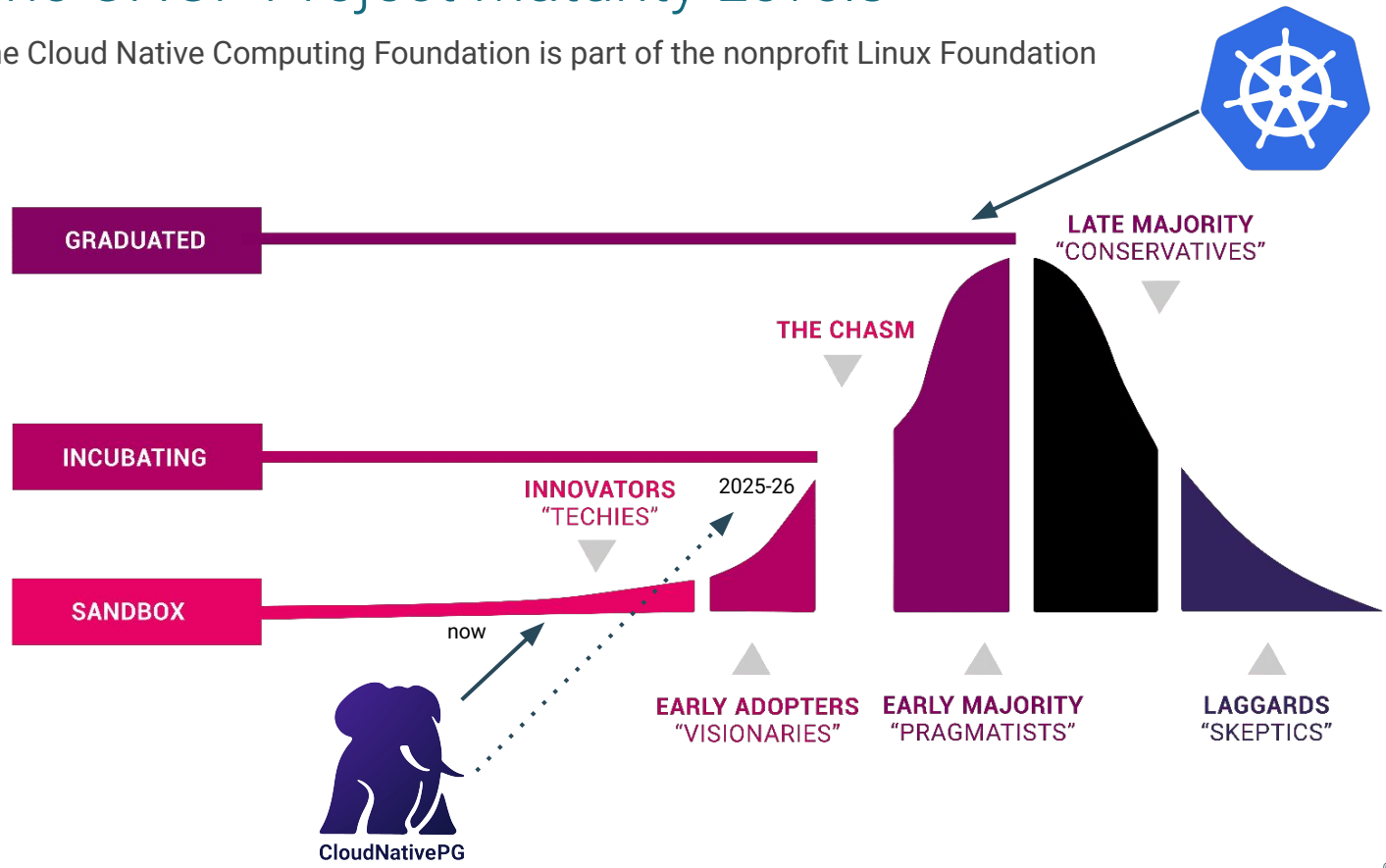
From Handcrafted PostgreSQL to Cloud-Neutral Automation with GitOps & K8s

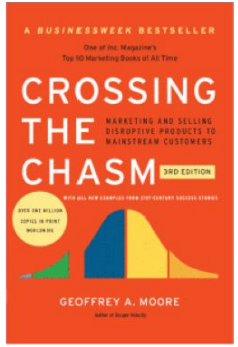


©EDB 2024 — ALL RIGHTS RESERVED.

The CNCF Project Maturity Levels

The Cloud Native Computing Foundation is part of the nonprofit Linux Foundation





©EDB 2024 — ALL RIGHTS RESERVED.

Trusted by industry leaders worldwide

Organizations running CloudNativePG in production

NAV

Nokia

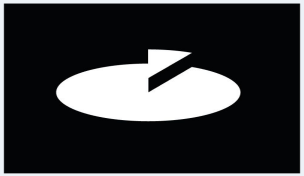
Telenor

Tesla

Novo Nordisk

Vestas

JN Data



Skatteetaten

©EDB 2024 — ALL RIGHTS RESERVED.

CNPG Operator

Postgres on Kubernetes

©EDB 2025 – ALL RIGHTS RESERVED.

Kubernetes timeline

- 2014, June: Google open sources Kubernetes
- 2015, July: Version 1.0 is released
- 2015, July: Google and Linux Foundation start the CNCF
- 2016, November: The operator pattern is introduced in a blog post
- 2018, August: The Community takes the lead
- 2019, April: Version 1.14 introduces Local Persistent Volumes
- 2019, August: EDB team starts the Kubernetes initiative
- 2020, June: we publish this blog about benchmarking local PVs on bare metal
- 2020, June: Data on Kubernetes Community founded
- 2021, February: EDB Cloud Native Postgres (CNP) 1.0 released
- 2022, May: EDB donates CNP and open sources it under CloudNativePG
- 2025, January: CloudNativePG was recognized as an official #CNCF project

©EDB 2025 — ALL RIGHTS RESERVED.

A kubernetes operator for Postgres

Kubernetes adoption is rising and it is already the de facto standard orchestration tool

PostgreSQL clusters “management the kubernetes way” enables many cloud native usage patterns, e.g. spinning up, disposable clusters during tests, one cluster per microservice and one database per cluster

CNPG tries to encode years of experience managing PostgreSQL clusters into an Operator which should automate all the known tasks a user could be willing to do

Our PostgreSQL operator must simulate a part of the work of a DBA

©EDB 2025 — ALL RIGHTS RESERVED.

Autopilot

It automates the steps that a human operator would do:

- to deploy
- to manage

©EDB 2025 – ALL RIGHTS RESERVED.

Security

CloudNativePG is secured by default:

- Automated encryption
- Principle of Least Privilege
- Native Declarative Connection Pooling
- Secure Database Authentication
- Automated "Day-2" Security Patches

©EDB 2025 – ALL RIGHTS RESERVED.

It doesn’t rely on statefulsets and uses its own way to manage persistent volume claims where the PGDATA is stored.

Data persistence

©EDB 2025 – ALL RIGHTS RESERVED.

Designed for Kubernetes

- entirely **Declarative**
- **Integrates** with the Kubernetes API server
- **No external tool** for management tool.

©EDB 2025 – ALL RIGHTS RESERVED.

Features

Deployment	Administration	Backup & Recovery	Monitoring	Security	High Availability
Kubernetes operator	Single node	Backup	Prometheus	TDE	Switchover
Kubernetes plugin	Cluster (Multi node)	Recovery	Grafana dashboards	Certificates	Failover
EDB Postgres (EPAS)	PostgreSQL configuration	PITR	Postgres Enterprise Manager	Data redaction	Scale out / scale down
PostGIS	Pooling	Volume Snapshots	Logging	Password management	Minor / Major updates

©EDB 2025 – ALL RIGHTS RESERVED.

What CNPG gives you : Day 1 and Day 2

CloudNativePG automates what a DBA would do; deploy, manage, heal, back up, upgrade; all through Kubernetes-native YAML.

Day 1 — get it running

Operator + OLM plugin
Install from OperatorHub in one click

Single-node or HA cluster
Declare instances: 1 or instances: 3 in YAML

PostgreSQL config as code
All pg parameters live in the Cluster manifest

EPAS, PostGIS, pgvector
Swap image tag to change Postgres flavour

TLS certificates out of the box
Mutual TLS between all cluster components

Day 2 — keep it healthy

Auto failover in < 30 seconds
Lease-based fencing prevents split-brain

Backup + PITR to object storage
Barman Cloud to Minio, S3, GCS, or Azure

Prometheus + Grafana built-in
~200 metrics, no extra exporter to install

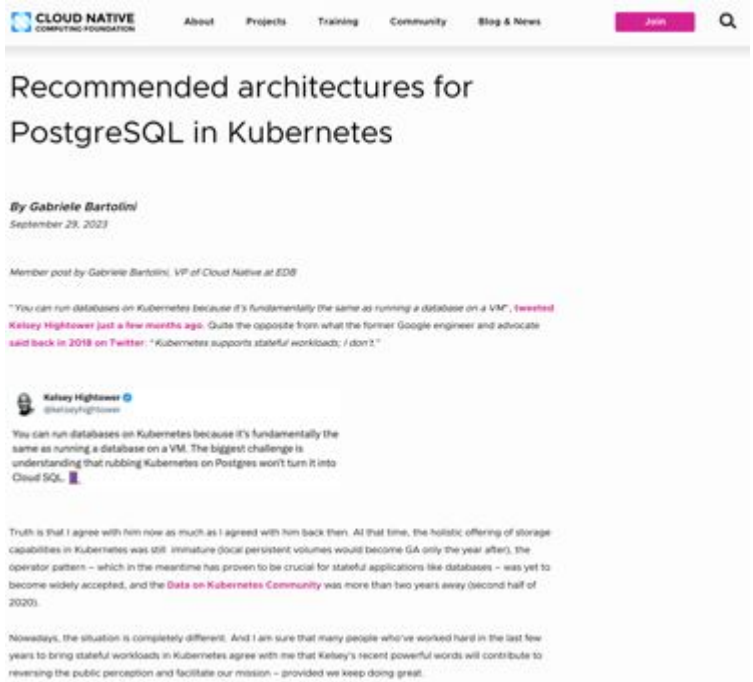
Minor + major version upgrades
Change image tag — operator handles pg_upgrade

PgBouncer connection pooling
Declare a Pooler CRD — operator manages it

©EDB 2025 – ALL RIGHTS RESERVED.

Recommended architectures

https://www.cncf.io/blog/2023/09/29/recommended-architectures-for-postgresql-in-kubernetes/



CLOUD NATIVE COMPUTING FOUNDATION

About Projects Training Community Blog & News Join

Recommended architectures for PostgreSQL in Kubernetes

By Gabriele Bartolini
September 29, 2023

Member post by Gabriele Bartolini, VP of Cloud Native at EDB

"*You can run databases on Kubernetes because it's fundamentally the same as running a database on a VM*", tweeted Kelsey Hightower just a few months ago. Quite the opposite from what the former Google engineer and advocate said back in 2018 on Twitter: "*Kubernetes supports stateful workloads; I don't.*"

Kelsey Hightower
@kelseyhightower

You can run databases on Kubernetes because it's fundamentally the same as running a database on a VM. The biggest challenge is understanding that rubbing Kubernetes on Postgres won't turn it into Cloud SQL.

Truth is that I agree with him now as much as I agreed with him back then. At that time, the holistic offering of storage capabilities in Kubernetes was still immature (local persistent volumes would become GA only the year after), the operator pattern – which in the meantime has proven to be crucial for stateful applications like databases – was yet to become widely accepted, and the Data on Kubernetes Community was more than two years away (second half of 2020).

Nowadays, the situation is completely different. And I am sure that many people who've worked hard in the last few years to bring stateful workloads in Kubernetes agree with me that Kelsey's recent powerful words will contribute to reversing the public perception and facilitate our mission – provided we keep doing great.

©EDB 2025 — ALL RIGHTS RESERVED.

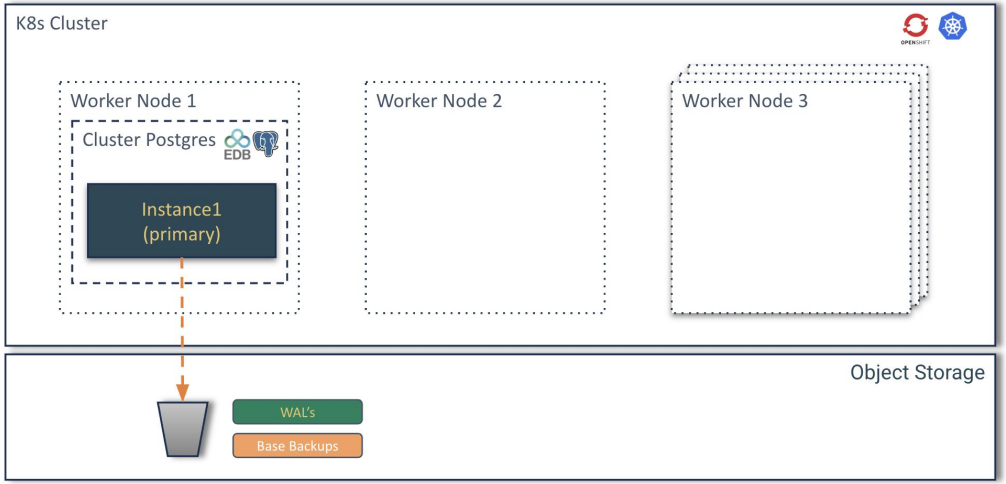
Use Cases

©EDB 2025 – ALL RIGHTS RESERVED.

Use case 1 architecture

A single database is the simplest setup, involving one instance of a database server.

- Development and testing environments
- Small applications with low traffic
- Non-critical data analysis
- Applications with high tolerance for downtime
- Cost-sensitive projects

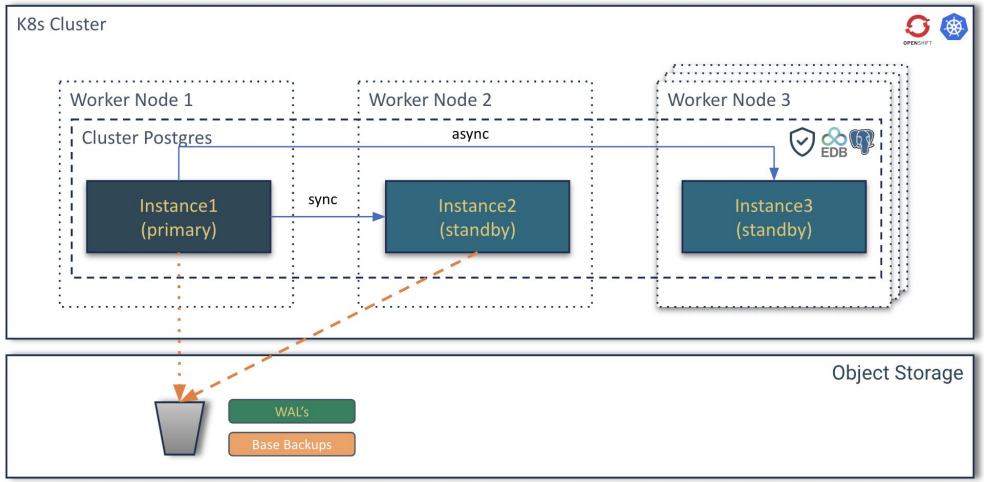


©EDB 2025 — ALL RIGHTS RESERVED.

Use case 2 architecture

An HA database setup aims to minimize downtime by having redundant components. If one component fails, another takes over automatically or with minimal intervention. This usually involves techniques like clustering, replication, or mirroring **within the same data center or availability zone.**

- Business critical Applications
- Applications with stringent SLAs
- Real-time systems
- Improving user experience
- Minimizing planned downtime

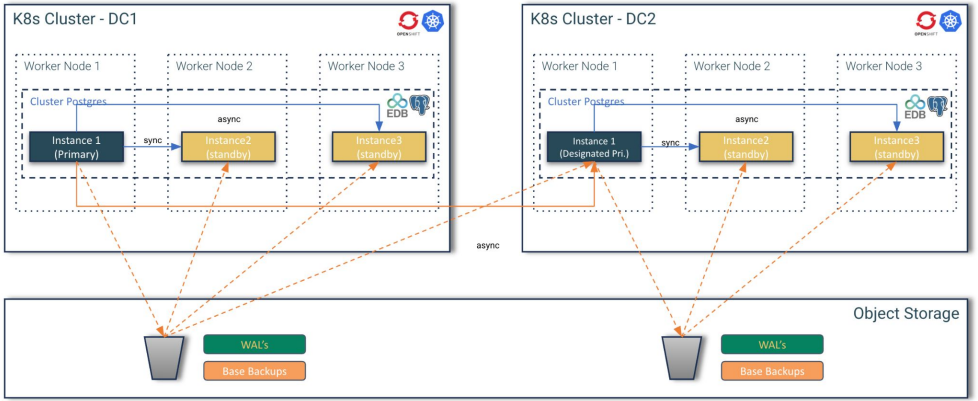


©EDB 2025 — ALL RIGHTS RESERVED.

Use case 3 architecture

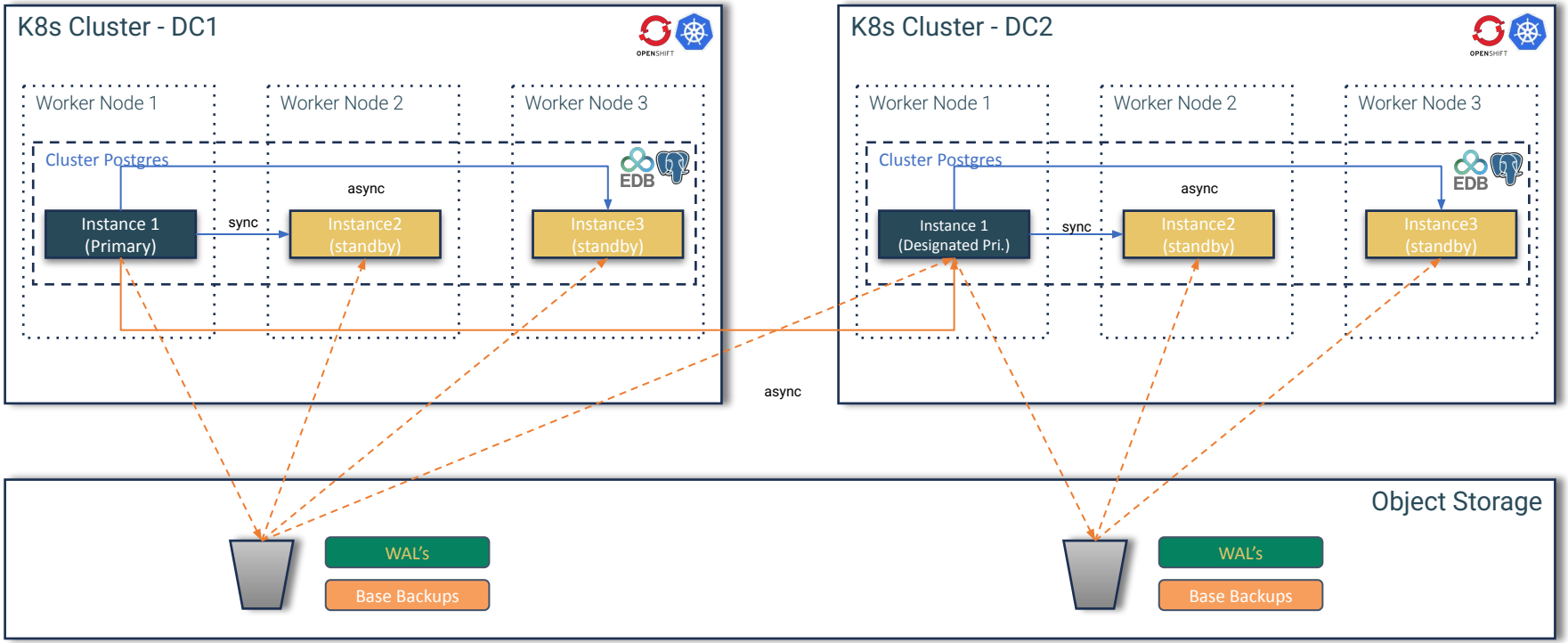
A DR database setup focuses on protecting data and ensuring business continuity in the event of a large-scale disaster affecting an entire data center or region (e.g., natural disasters, power outages, cyberattacks). This typically involves replicating data to a geographically separate location.

- Regulatory compliance
- Protecting against catastrophic data loss
- Ensuring business continuity for mission-critical systems



©EDB 2025 — ALL RIGHTS RESERVED.

Use case 3 architecture



©EDB 2025 — ALL RIGHTS RESERVED.

The Workshop

©EDB 2025 — ALL RIGHTS RESERVED.

What you will do today

- **Kubernetes plugin install**
- **Check the CloudNativePG operator status**
- **Postgres cluster install**
- **Insert data in the cluster**
- **Failover**
- **Backup**
- **Recovery**
- **Scale out/down**
- **Fencing**
- **Hibernation**

What has been done

- **CNPG operator installation**
- **Kubernetes plugin install `kubectl-cnp`**
- **S3 bucket creation**

Last CloudNativePG tested version is 1.25

©EDB 2025 – ALL RIGHTS RESERVED.

Workshop architecture

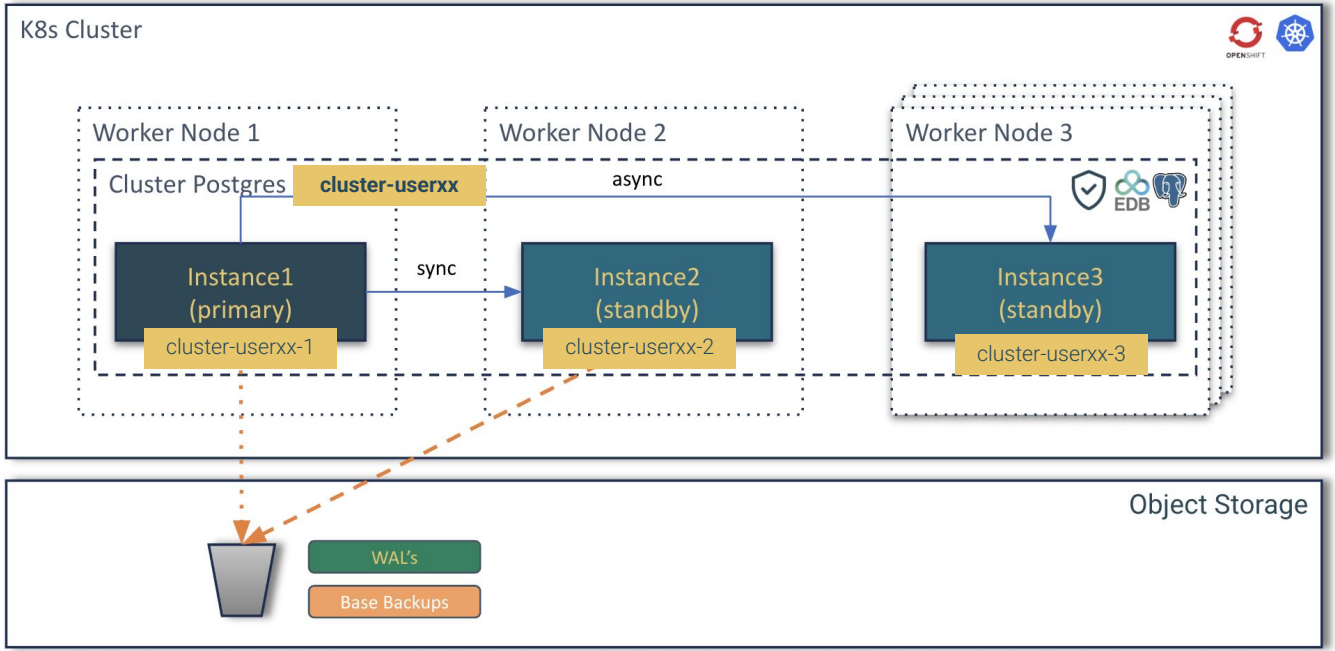
Name space's PG cluster
edb-emea-userXX

Services

- cluster-userxx-rw
- cluster-userxx-ro
- cluster-userxx-rw

Name space's S3 storage

minio



©EDB 2025 — ALL RIGHTS RESERVED.

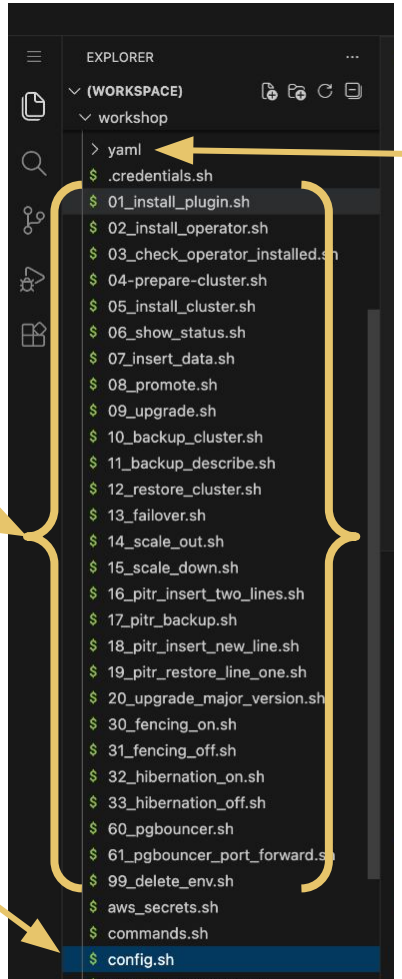
How the project is build

You will run script to do all the task:

- 03_check_operator.sh
- 04_prepare_cluster.sh
- 05_install_cluster.sh
- …

Variable are stored in

- config.sh



Custom Ressources are in YAML folder

They are call by scripts .sh

©EDB 2025 — ALL RIGHTS RESERVED.

Example of a script

```
kubectl-cnp apply -n edb-emae-userxx -f ./yaml/cluster-sample.yaml
```

05_install_cluster.sh

```
#!/bin/bash
. ./config.sh
. ./env.sh

#Doc
echo "05" > ./docs/docid

if [[ $? -eq 1 ]]; then
   echo "Error occurred. Please fix the problem."
   exit 1
fi

# Create namespace if does not exists
. ./create_namespace.sh

print_command "${kubectl_cmd} apply -n ${namespace}
-f ./yaml/cluster-sample.yaml\n"

envsubst < ./yaml/cluster-sample.yaml |
${kubectl_cmd} apply -n ${namespace} -f-
```

config.sh

```
# Kubernetes environment configuration
export namespace="edb-${region}-${id}"
export kubectl_cmd="oc"
export kubectl_cnp="kubectl-cnp"
...
```

cluster-sample.yaml

```
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: $cluster_name
spec:
 instances: $postgres_instances
 logLevel: info
 imageName: "$postgres_default_image"
 imagePullSecrets:
...
```

©EDB 2025 – ALL RIGHTS RESERVED.

Interactive session
It’s time to go hands-on!

©EDB 2025 – ALL RIGHTS RESERVED.

Hand-on documentation

Download this presentation

http://bit.ly/4dEQwqc

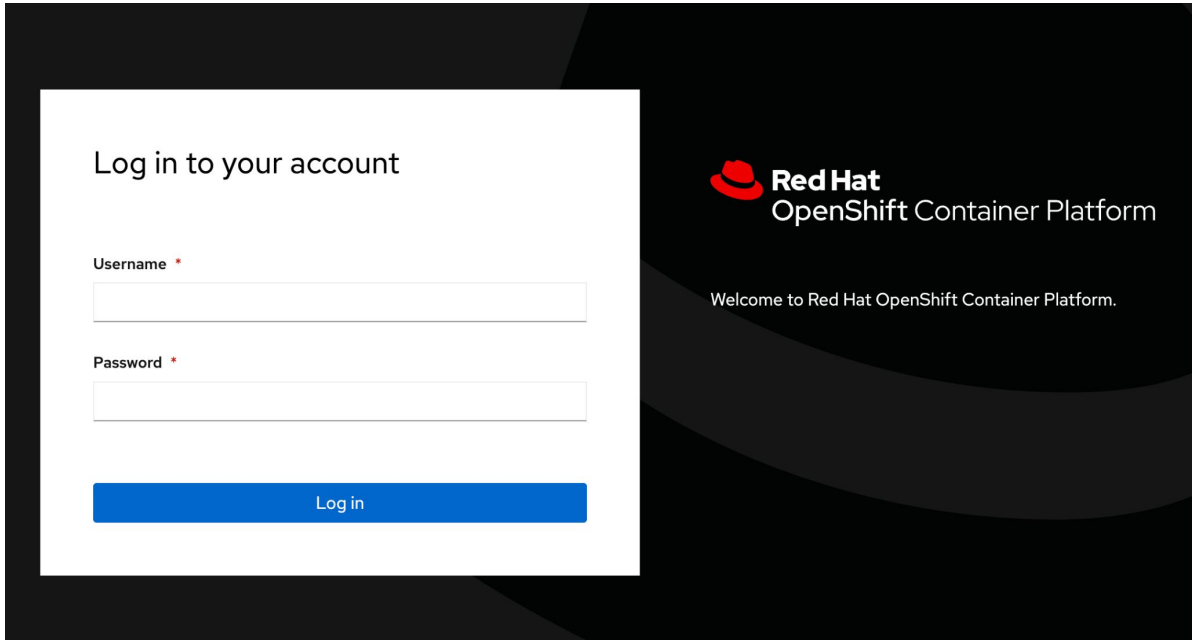
©EDB 2024 — ALL RIGHTS RESERVED.

Environment

OpenShift Console	https://console-openshift-console.apps.ocp.t8s7c.sandbox5452.opentlc.com
OpenShift User name	user01..user30
OpenShift Password	EDB-RH-PASS26
Devspace url:	https://devspaces.apps.ocp.t8s7c.sandbox5452.opentlc.com/
Minio UI	https://minio-ui-minio.apps.ocp.t8s7c.sandbox5452.opentlc.com
Minio API	https://minio-api-minio.apps.ocp.t8s7c.sandbox5452.opentlc.com
Minio User	minio
Minio password	edb-workshop
Github create OpenShit env	https://github.com/michael-bang/devspace-edb
Github create Postgres demo	https://github.com/sergioenterprisedb/edb-postgres-for-kubernetes-in-openshift/tree/2026-aarhus

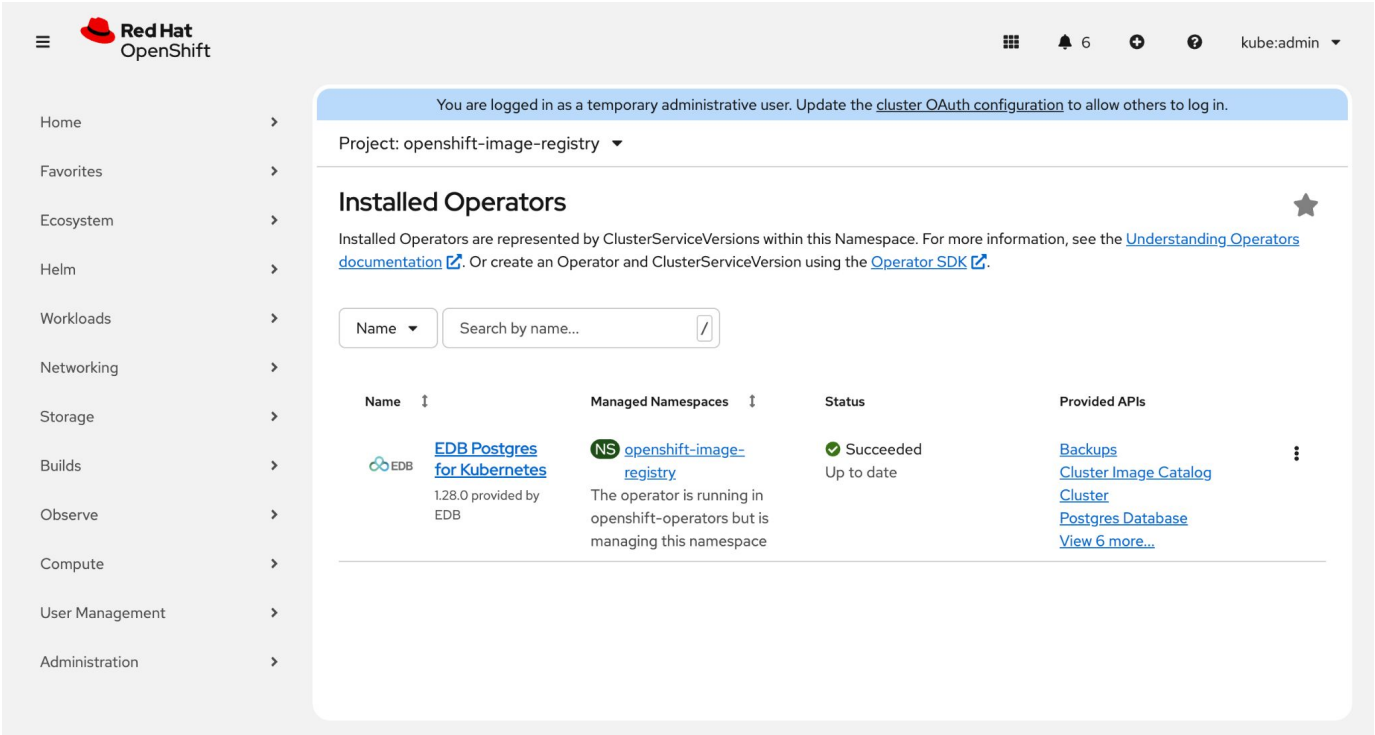
©EDB 2025 — ALL RIGHTS RESERVED.

Red Hat OpenShift Login



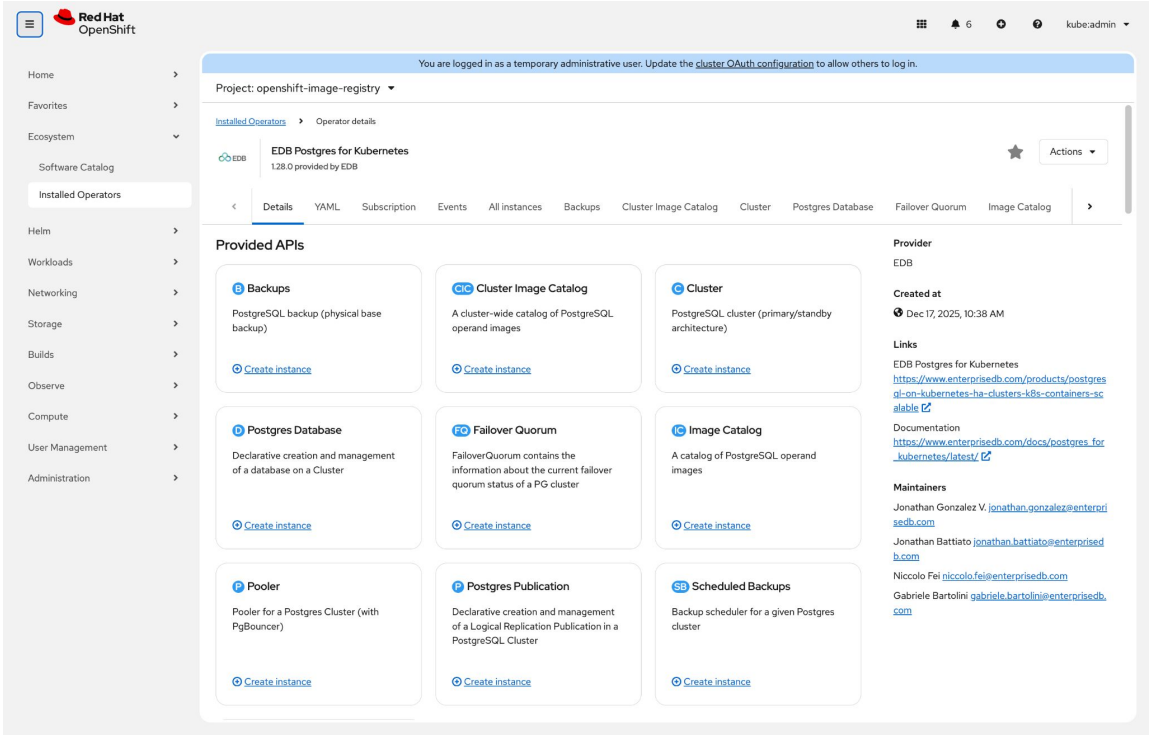
©EDB 2025 — ALL RIGHTS RESERVED.

Red Hat OpenShift environment



©EDB 2025 — ALL RIGHTS RESERVED.

Red Hat OpenShift operator



©EDB 2025 — ALL RIGHTS RESERVED.

Hands-on

©EDB 2025 — ALL RIGHTS RESERVED.

Plugging and OpenShift login

Scripts

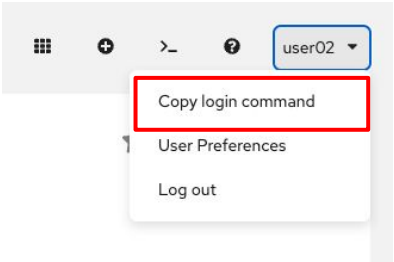
`01_install_plugin.sh`

Install plugin

CloudNativePG provides a plugin for kubectl to manage a Postgres cluster in Kubernetes. You can manage status, promote, certificates, restart, reload, maintenance, report, logs, destroy, hibernation, backups, …

Doc link

OpenShift Login and namespace creatop,



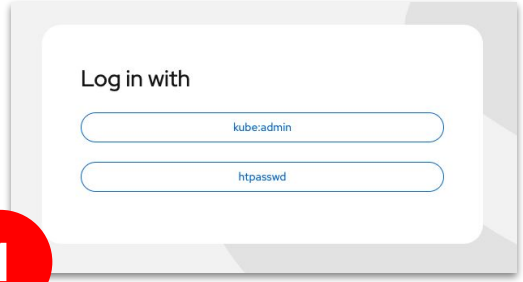
Code

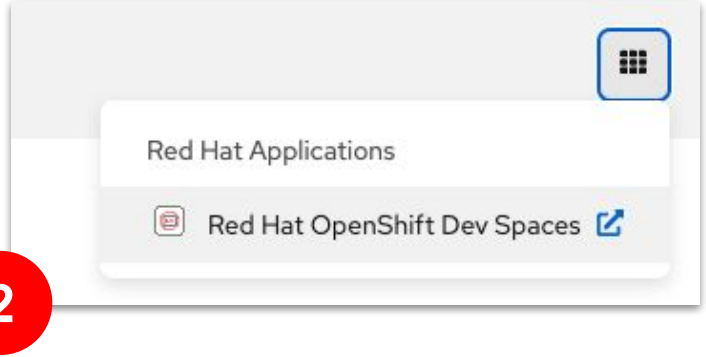
```
# Install plugin

curl -sSfL \
  https://github.com/EnterpriseDB/kubectl-cnp/raw/main/install.sh
| sh -s -- -b .


# Login to the OpenShift Cluster

oc login ...

# Create your namespace

oc new-project userXX
```

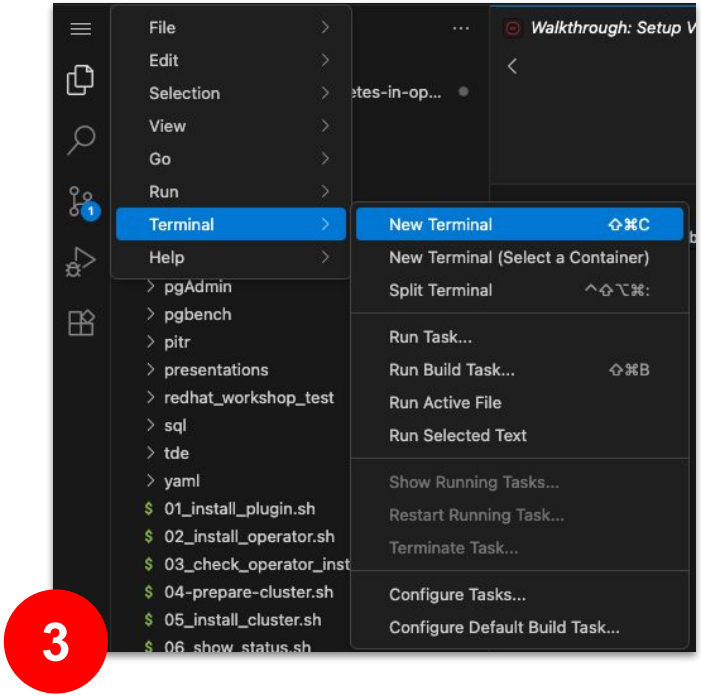
©EDB 2025 — ALL RIGHTS RESERVED.

OpenShift and DevSpaces access

https://github.com/raphael-chir/edb-postgres-for-kubernetes-in-openshift







©EDB 2025 — ALL RIGHTS RESERVED.

Checking operator installation

Scripts

`03_check_operator_installed.sh`

Checking operator installation

In Kubernetes, the operator is by default installed in the postgresql-operator-system namespace as a Kubernetes Deployment. The name of this deployment depends on the installation method. When installed through the manifest or the cnp plugin, by default, it is called postgresql-operator-controller-manager. When installed via Helm, by default, the deployment name is derived from the helm release name, appended with the suffix -edb-postgres-for-kubernetes (e.g., <name>-edb-postgres-for-kubernetes).

Doc link

Code

```
# Check operator installation

oc get deploy -A | grep postgres
NAMESPACE:      openshift-operators
NAME:           postgresql-operator-controller-manager
READY:          1/1
UP-TO-DATE:     1
AVAILABLE:      1
AGE:            5d21h
```

©EDB 2025 — ALL RIGHTS RESERVED.

Install Postgres cluster

Scripts

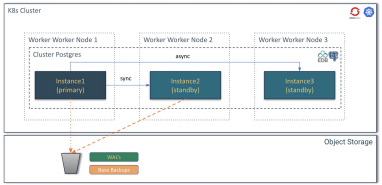
```
04-prepare-cluster.sh
05_install_cluster.sh
06_show_status.sh
07_insert_data.sh
```

Prepare Postgres cluster template

Setup credentials (AWS or Minio) and prepare context for your users

Deploy Postgres cluster in HA

Different types of architectures can be deployed. In our demo, 3 instances are deployed with HA available.



Insert Data

Insert data in `test` table

Doc link

Code

```
export cluster_name="cluster-userXX"
export TMP=$TMPDIR

# Get template
envsubst < templates/${cluster_name}-template.yaml >
$TMP/${cluster_name}.yaml

# Deploy Postgres cluster
${kubectl_cmd} apply -f $TMP/${cluster_name}.yaml
---------------------------------------------------------------------------
ApiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: cluster-example
spec:
 instances: 3
 imageName: "docker.enterprisedb.com/k8s/edb-postgres-advanced:17.6"
 enableSuperuserAccess: true

 replicationSlots:
   highAvailability:
     enabled: true
```

©EDB 2025 — ALL RIGHTS RESERVED.

Monitoring

Show status

```
Cluster Summary
Name                     default/cluster-sergio-1
System ID:               7566239054312464401
PostgreSQL Image:        docker.enterprisedb.com/k8s/edb-postgres-advanced:17.6
Primary instance:        cluster-sergio-1
Primary promotion time:  2025-10-28 11:52:15 +0000 UTC (4m30s)
Status:                  Cluster in healthy state
Instances:               3
Ready instances:         3
Size:                    127M
Current Write LSN:       0/6032300 (Timeline: 1 - WAL File: 000000010000000000000006)

Continuous Backup status (Barman Cloud Plugin)
ObjectStore / Server name:      object-store/cluster-sergio
First Point of Recoverability:  -
Last Successful Backup:         -
Last Failed Backup:             -
Working WAL archiving:          OK
WALs waiting to be archived:    0
Last Archived WAL:              000000010000000000000005.00000028.backup   @   2025-10-28T11:54:11.908195Z
Last Failed WAL:                -

Streaming Replication status
Replication Slots Enabled
Name              Sent LSN   Write LSN  Flush LSN  Replay LSN  Write Lag  Flush Lag  Replay Lag  State      Sync State  Sync Priority  Replication Slot
----              --------   ---------  ---------  ----------  ---------  ---------  ----------  -----      ----------  -------------  ----------------
cluster-sergio-2  0/9000000  0/9000000  0/9000000  0/9000000   00:00:00   00:00:00   00:00:00    streaming  quorum      1              active
cluster-sergio-3  0/9000000  0/9000000  0/9000000  0/9000000   00:00:00   00:00:00   00:00:00    streaming  quorum      1              active

Instances status
Name              Current LSN  Replication role  Status  QoS         Manager Version  Node
----              -----------  ----------------  ------  ---         ---------------  ----
cluster-sergio-1  0/9000000    Primary           OK      Guaranteed  1.28.0           ocp4-multiarch-worker1-ppc64
cluster-sergio-2  0/9000000    Standby (sync)    OK      Guaranteed  1.28.0           ocp4-multiarch-w5gg4-master-2
cluster-sergio-3  0/9000000    Standby (sync)    OK      Guaranteed  1.28.0           ocp4-multiarch-w5gg4-master-1
```

©EDB 2025 — ALL RIGHTS RESERVED.

Promote and Failover

Scripts

08_promote.sh

Promote

The meaning of this command is to promote a pod in the cluster to primary, so you can start with maintenance work or test a switch-over situation in your cluster:

Doc link

Code

```
# Promote

. ./config.sh

${kubectl_cnp} promote ${cluster_name} ${cluster-name}-2
```

Name of Cluster
Eg: cluster-userxx

Name of the 2nd pod
Eg: cluster-userxx-2

©EDB 2025 — ALL RIGHTS RESERVED.

Minor upgrade

Scripts

09_upgrade.sh

Minor upgrade

To upgrade to a newer minor version, simply update the PostgreSQL container image reference in your cluster definition, either directly or via image catalogs. CloudNativePG will trigger a rolling update of the cluster, replacing each instance one by one, starting with the replicas. Once all replicas have been updated, it will perform either a switchover or a restart of the primary to complete the process.

Doc link

Code

```
# Minor upgrade

# From
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: cluster-sergio1
spec:
 instances: 3
 imageName:
"docker.enterprisedb.com/k8s/edb-postgres-advanced:17.6"
...

# To
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: cluster-sergio1
spec:
 instances: 3
 imageName:
"docker.enterprisedb.com/k8s/edb-postgres-advanced::17.7"
...
```

©EDB 2025 — ALL RIGHTS RESERVED.

Backups

Scripts

```
10_backup_cluster.sh
11_backup_describe.sh
```

Backup

Backups are used using Barman cloud with the new Barman Plugin. In case the plugin is not compatible (like in Red Hat OpenShift), we will continue using Barman Cloud (without the plugin).

PostgreSQL natively provides first class backup and recovery capabilities based on file system level (physical) copy. These have been successfully used for more than 15 years in mission critical production databases, helping organizations all over the world achieve their disaster recovery goals with Postgres.

Doc link

Code

```
cat $TMP/backup.yaml

apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Backup
metadata:
 name: cluster-example-backup-test
spec:
 cluster:
   name: cluster-example

${kubectl_cmd} apply -f ./yaml/backup.yaml
```

©EDB 2025 — ALL RIGHTS RESERVED.

Recovery

Scripts

12_restore_cluster.sh

Recovery

In PostgreSQL, recovery refers to the process of starting an instance from an existing physical backup. PostgreSQL's recovery system is robust and feature-rich, supporting Point-In-Time Recovery (PITR)—the ability to restore a cluster to any specific moment, from the earliest available backup to the latest archived WAL file.

Doc link

Code

```
cat $TMP/restore.yaml

apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: cluster-restore
spec:
 instances: 1
 imageName: quay.io/enterprisedb/postgresql:16.4
 imagePullPolicy: IfNotPresent
 bootstrap:
   recovery:
     source: source
 plugins:
 - name: barman-cloud.cloudnative-pg.io
   isWALArchiver: true
   parameters:
     barmanObjectName: object-store
 externalClusters:
 - name: source
   plugin:
     name: barman-cloud.cloudnative-pg.io
     parameters:
       barmanObjectName: object-store
       serverName: cluster-example

${kubectl_cmd} apply -f $TMP/restore.yaml
```

©EDB 2025 — ALL RIGHTS RESERVED.

Promote and Failover

Scripts

`13_failover.sh`

Automated Failover

In the case of unexpected errors on the primary for longer than the `.spec.failoverDelay` (by default 0 seconds), the cluster will go into failover mode. This may happen, for example, when:

- The primary pod has a disk failure
- The primary pod is deleted
- The `postgres` container on the primary has any kind of sustained failure

In the failover scenario, the primary cannot be assumed to be working properly.
Doc link

Code

```
# Automatic Failover (simulate failover killing pods of primary)
${kubectl_cmd} delete pvc/${primary} \
                      pvc/${primary}-wal \
                      pod/${primary} \
                      --force
```

©EDB 2025 — ALL RIGHTS RESERVED.

Scale out, scale down

Scripts

```
14_scale_out.sh
15_scale_down.sh
```

Scale out and down

The operator allows you to scale up and down the number of instances in a PostgreSQL cluster. New replicas are started up from the primary server and participate in the cluster's HA infrastructure.

Doc link

Code

```
# Scale out
kubectl scale cluster cluster-example --replicas=4


# Scale down
kubectl scale cluster cluster-example --replicas=2
```

©EDB 2025 — ALL RIGHTS RESERVED.

Fencing and Hibernation

Scripts

```
30_fencing_on.sh
31_fencing_off.sh
```

Fencing

Fencing is the process of protecting the data in one, more, or even all instances of a PostgreSQL cluster when they appear to be malfunctioning. When an instance is fenced, the PostgreSQL server process is guaranteed to be shut down, while the pod is kept running. This ensures that, until the fence is lifted, data on the pod isn't modified by PostgreSQL and that you can investigate file system for debugging and troubleshooting purposes.

Doc link

Code

```
# Fencing on
${kubectl_cnp} fencing on ${cluster_name} ${replica}

# Fencing off
${kubectl_cnp} fencing off ${cluster_name} ${replica}
```

©EDB 2025 — ALL RIGHTS RESERVED.

Fencing and Hibernation

Scripts

```
32_hibernation_on.sh
33_hibernation_off.sh
```

Hibernation

CloudNativePG supports hibernation of a running PostgreSQL cluster in a declarative manner, through the cnpg.io/hibernation annotation. Hibernation enables saving CPU power by removing the database pods while keeping the database PVCs. This feature simulates scaling to 0 instances.

Doc link

Code

```
# Hibernation on
${kubectl_cmd} annotate cluster ${cluster_name} \
            --overwrite k8s.enterprisedb.io/hibernation=on

# Hibernation off
${kubectl_cmd} annotate cluster ${cluster_name} \
            --overwrite k8s.enterprisedb.io/hibernation=off
```

©EDB 2025 — ALL RIGHTS RESERVED.

Major upgrade (copying data)

Scripts

```
20_upgrade_major_version.sh
```

Major upgrade (with native logical replication)
Major PostgreSQL releases introduce changes to the internal data storage format, requiring a more structured upgrade process.

CloudNativePG supports three methods for performing major upgrades:

- Logical dump/restore – Blue/green deployment, offline.
- Native logical replication – Blue/green deployment, online.
- **Physical with pg_upgrade** – In-place upgrade, offline (covered in the "Offline In-Place Major Upgrades" section below).

Doc link

Code

```yaml
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: ${cluster_major_upgrade}
spec:
 instances: 1
 imageName: ${postgres_major_upgrade_image}

 storage:
   size: 2Gi

 bootstrap:
   initdb:
     import:
       type: microservice
       databases:
         - app
       source:
         externalCluster: ${cluster_name}

 externalClusters:
   - name: ${cluster_name}
     connectionParameters:
       # Use the correct IP or host name for the source database
       host: ${cluster_name}-rw
       user: postgres
       dbname: postgres
     password:
       name: ${cluster_name}-superuser
       key: password
```

©EDB 2025 — ALL RIGHTS RESERVED.

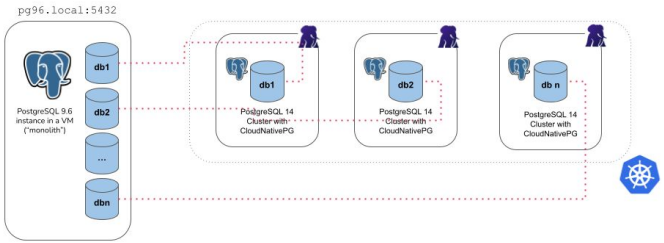
Major upgrade (copying data)

Scripts

`20_upgrade_major_version.sh`

Major upgrade (with native logical replication)

- In this step we will migrate the app database from our existing cluster to the new cluster
- We will create the yaml file with the setting "**import**" in the bootstrap section
- The operator uses internally postgres tools pg_dump and pg_restore
- This method can be used to **migrate** another database or to run **out-of-the-place upgrade**
- Possible settings:
  - Microservices
  - Monolith



Code

```yaml
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: cluster-sergio-major
spec:
 instances: 1
 imageName: docker.enterprisedb.com/k8s/postgresql:18.1
 imagePullSecrets:
 - name: postgresql-operator-pull-secret
 enableSuperuserAccess: true

 bootstrap:
   initdb:
     import:
       type: microservice
       databases:
         - postgres
       source:
         externalCluster: cluster-sergio

 externalClusters:
   - name: cluster-sergio
     connectionParameters:
       host: cluster-sergio-rw
       user: postgres
       dbname: postgres
     password:
       name: cluster-sergio-superuser
       key: password
...
```

©EDB 2025 — ALL RIGHTS RESERVED.

Thank you

For any queries, contact:
stephen.hall@enterprisedb.com

Backup Slides

©EDB 2025 – ALL RIGHTS RESERVED.

Architectures

©EDB 2025 – ALL RIGHTS RESERVED.

When to choose Kubernetes over VMs?

01 | Cloud Native Applications that already run in Kubernetes

02 | Scalable, replicated databases

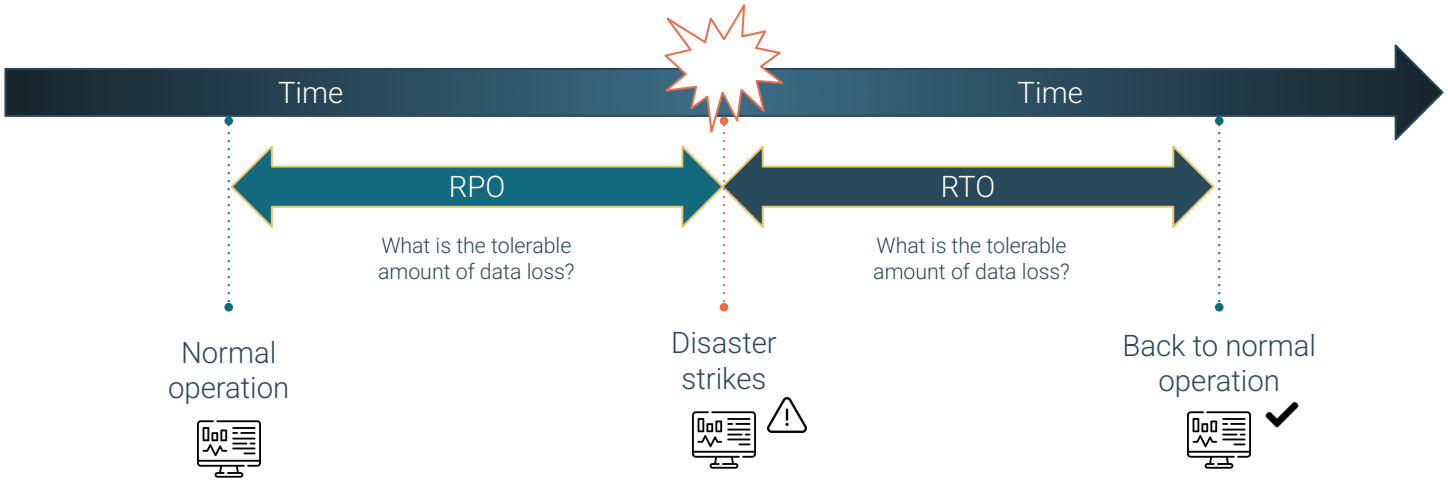
03 | Applications requiring automated failover and self-healing

04 | Teams skilled in Kubernetes who want a unified infrastructure

©EDB 2024 — ALL RIGHTS RESERVED.

Concepts

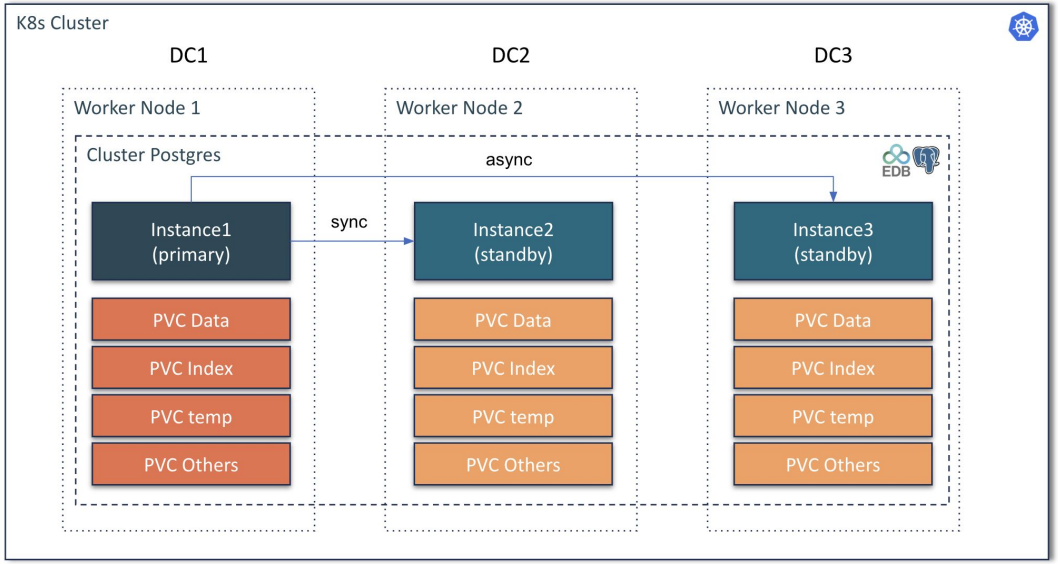
- Recovery Point Objective (**RPO**) and Recovery Time Objective (**RTO**) are key concepts in disaster recovery and business continuity planning, particularly related to data loss and system downtime.



©EDB 2025 — ALL RIGHTS RESERVED.

Red Hat Recommendation

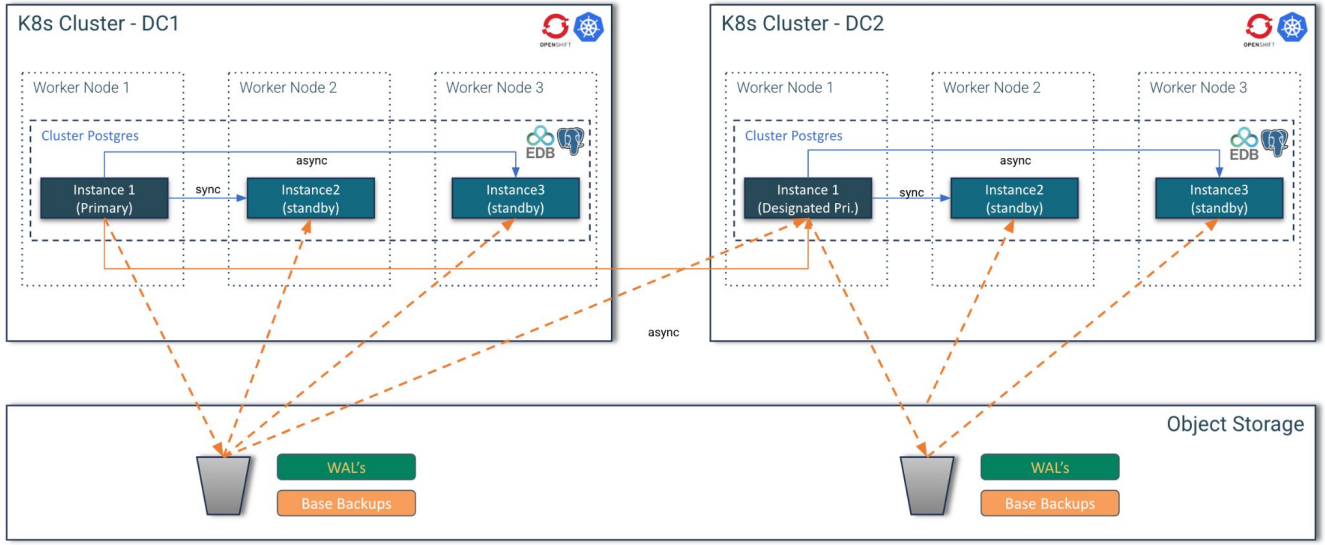
Red Hat recommend stretched clusters ONLY when latencies don't exceed 5 milliseconds (ms) round-trip time (RTT) between the nodes in different locations, with a maximum RTT of 10 ms.



©EDB 2025 — ALL RIGHTS RESERVED.

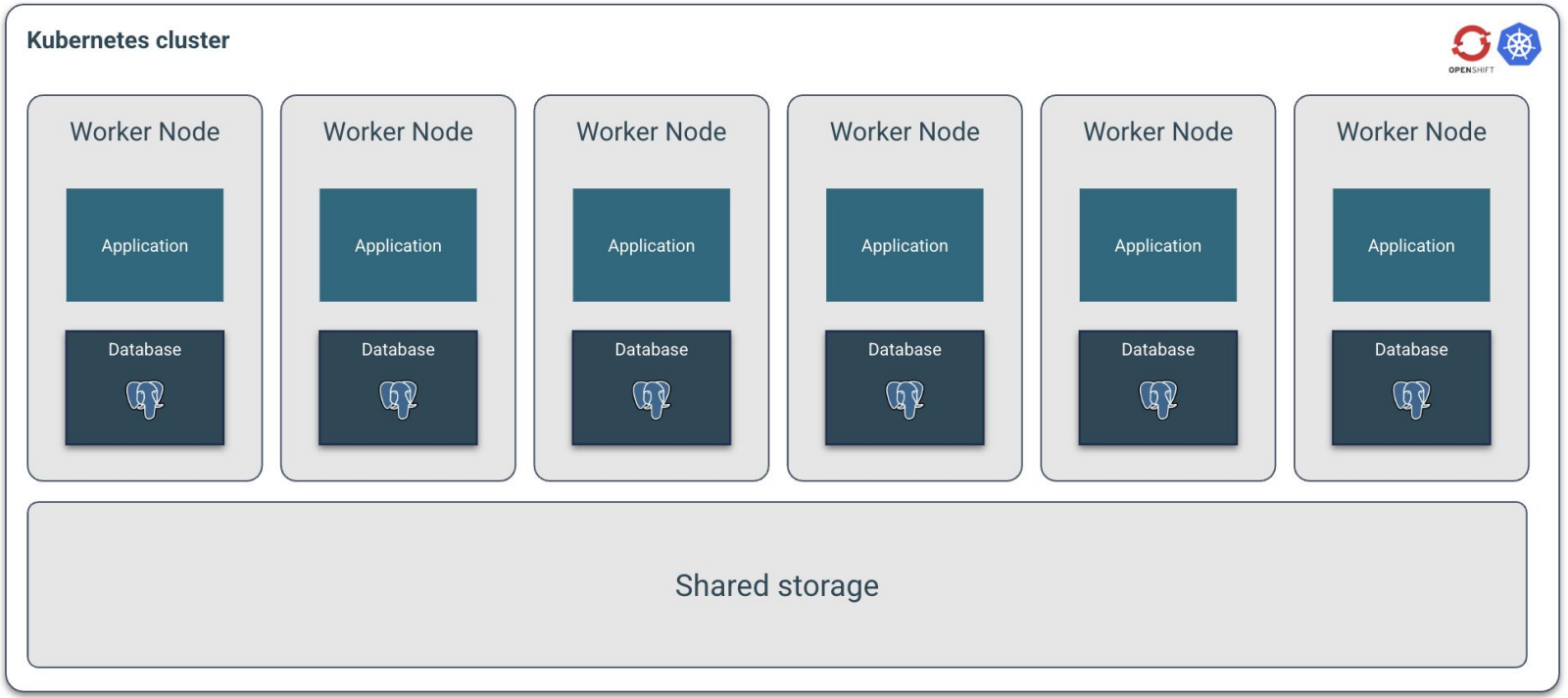
Two separate single data center Kubernetes clusters

In case you cannot go beyond two data centers and you end up with two separate Kubernetes clusters, don't despair.



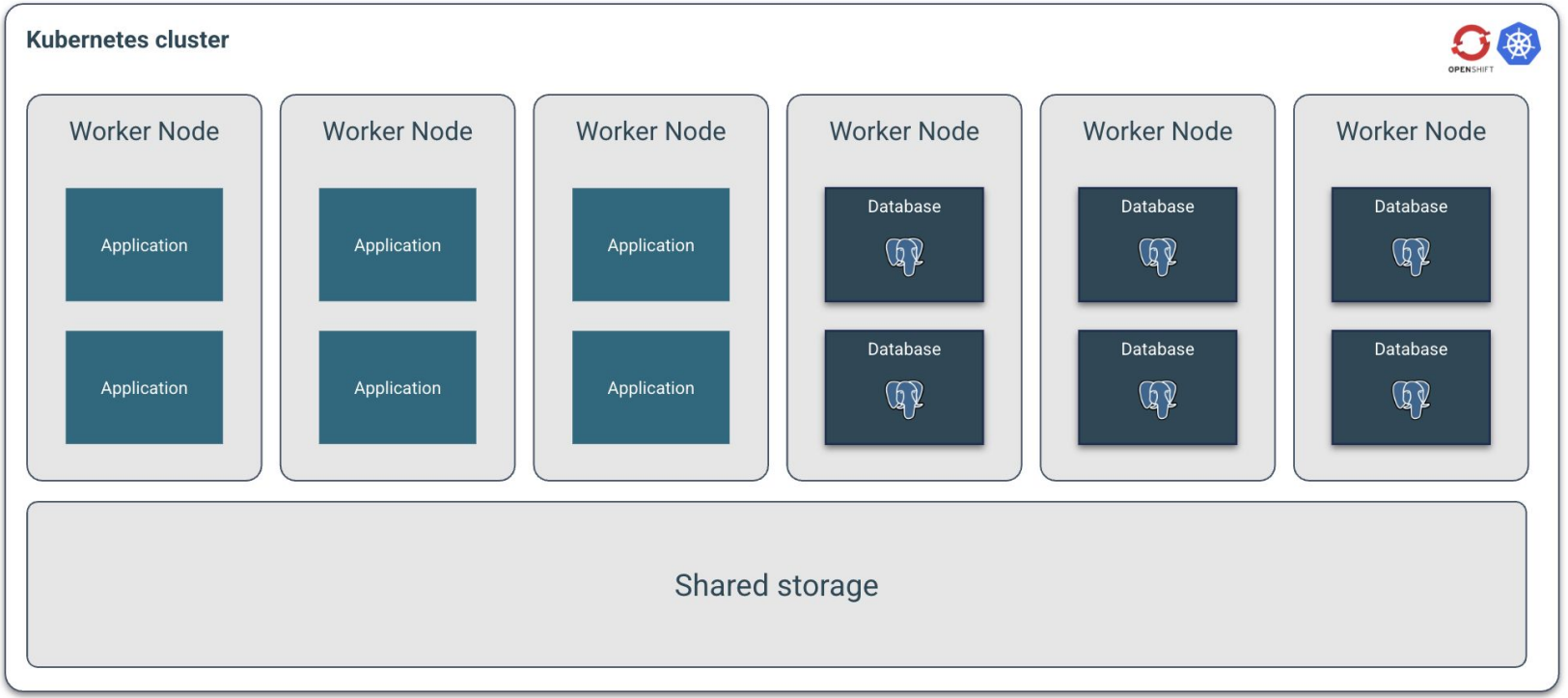
©EDB 2025 — ALL RIGHTS RESERVED.

Shared workload, shared storage



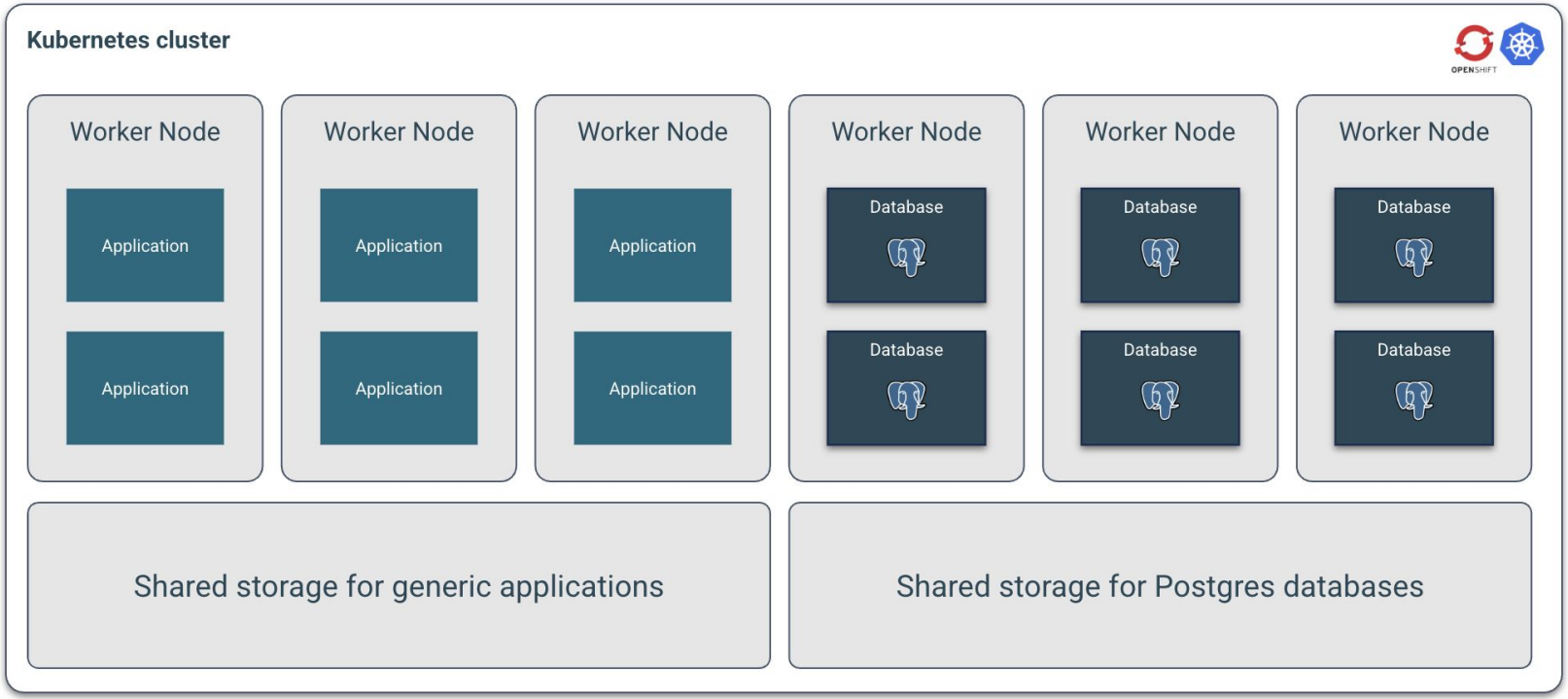
©EDB 2025 — ALL RIGHTS RESERVED.

Shared workload, shared storage



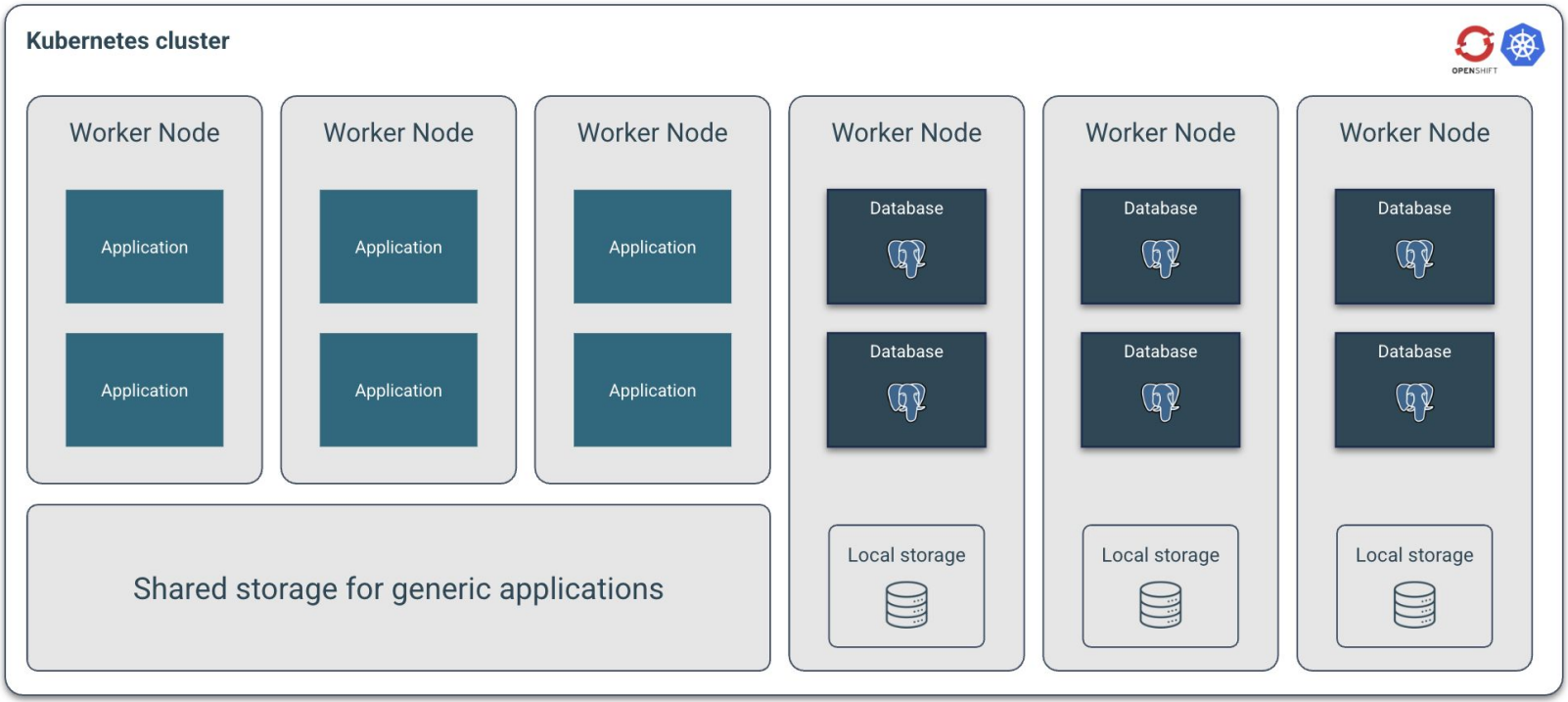
©EDB 2025 — ALL RIGHTS RESERVED.

Shared workload, shared storage



©EDB 2025 — ALL RIGHTS RESERVED.

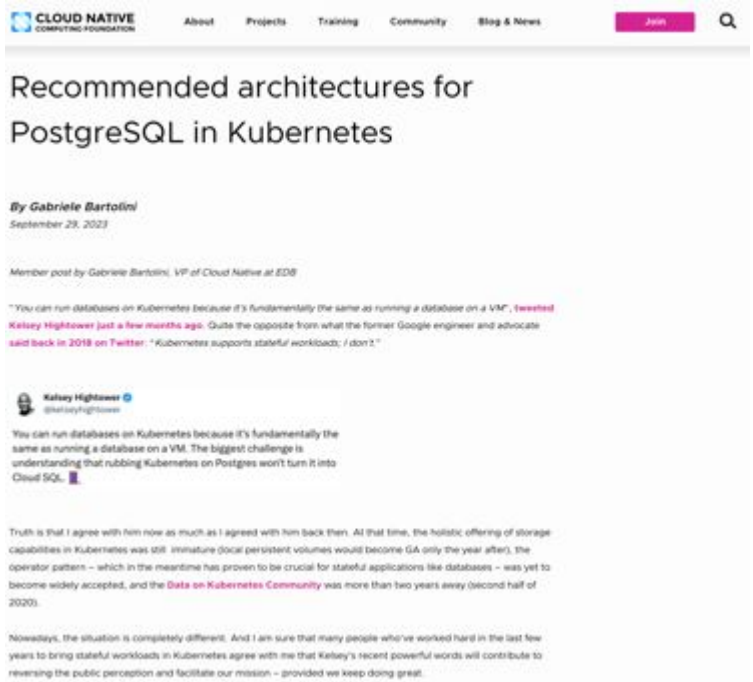
Shared workloads, local storage



©EDB 2025 — ALL RIGHTS RESERVED.

Recommended architectures

https://www.cncf.io/blog/2023/09/29/recommended-architectures-for-postgresql-in-kubernetes/



CLOUD NATIVE COMPUTING FOUNDATION

About Projects Training Community Blog & News Join

Recommended architectures for PostgreSQL in Kubernetes

By Gabriele Bartolini
September 29, 2023

Member post by Gabriele Bartolini, VP of Cloud Native at EDB

"*You can run databases on Kubernetes because it's fundamentally the same as running a database on a VM*", tweeted Kelsey Hightower just a few months ago. Quite the opposite from what the former Google engineer and advocate said back in 2018 on Twitter: "*Kubernetes supports stateful workloads; I don't.*"

Kelsey Hightower
@kelseyhightower

You can run databases on Kubernetes because it's fundamentally the same as running a database on a VM. The biggest challenge is understanding that rubbing Kubernetes on Postgres won't turn it into Cloud SQL.

Truth is that I agree with him now as much as I agreed with him back then. At that time, the holistic offering of storage capabilities in Kubernetes was still immature (local persistent volumes would become GA only the year after), the operator pattern – which in the meantime has proven to be crucial for stateful applications like databases – was yet to become widely accepted, and the Data on Kubernetes Community was more than two years away (second half of 2020).

Nowadays, the situation is completely different. And I am sure that many people who've worked hard in the last few years to bring stateful workloads in Kubernetes agree with me that Kelsey's recent powerful words will contribute to reversing the public perception and facilitate our mission – provided we keep doing great.

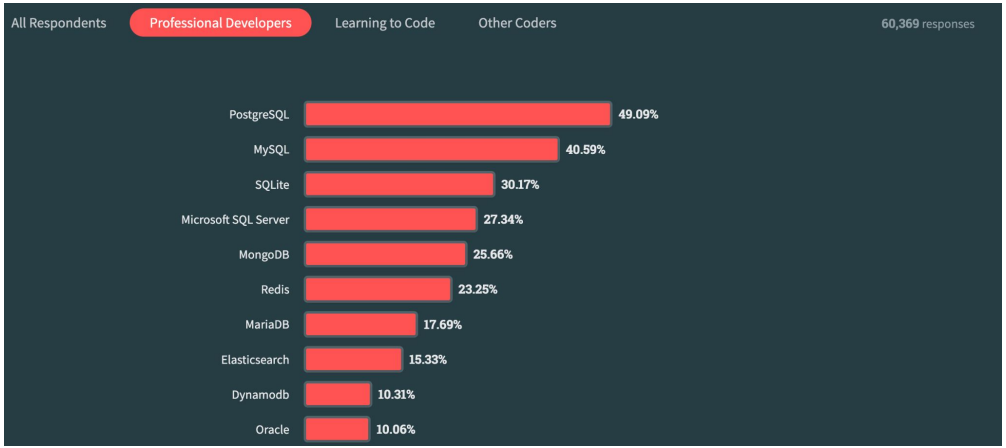
©EDB 2025 — ALL RIGHTS RESERVED.

PostgreSQL most loved database (2023)

Source: Stackoverflow

This year, PostgreSQL took over the first place spot from MySQL. Professional Developers are more likely than those learning to code to use PostgreSQL (50%) and those learning are more likely to use MySQL (54%).

MongoDB is used by a similar percentage of both Professional Developers and those learning to code and it's the second most popular database for those learning to code (behind MySQL).

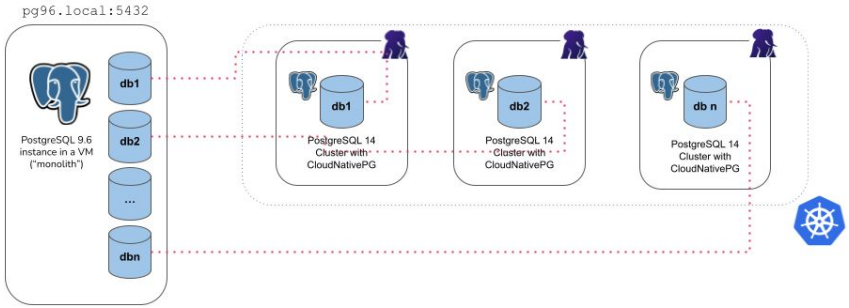


©EDB 2024 — ALL RIGHTS RESERVED.

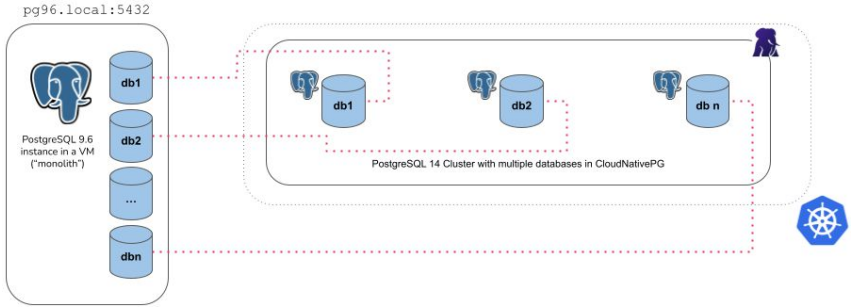
Database Migration

- In this step we will migrate the app database from our existing cluster to the new cluster
- We will create the yaml file with the setting "**import**" in the bootstrap section
- The operator uses internally postgres tools pg_dump and pg_restore
- This method can be used to **migrate** another database or to run **out-of-the-place upgrade**
- Possible settings:

Microservices:



Monolith:



©EDB 2025 – ALL RIGHTS RESERVED.

Major upgrade (copying data)

Scripts

```
24_major_upgrade_in_place.sh
25_verify_major_upgrade_16_17.sh
```

Major upgrade (offline In-Place Major Upgrades)

CloudNativePG performs an offline in-place major upgrade when a new operand container image with a higher PostgreSQL major version is declaratively requested for a cluster.

Major upgrades are only supported between images based on the same operating system distribution. For example, if your previous version uses a bullseye image, you cannot upgrade to a bookworm image.

Doc link

Code

```
apiVersion: postgresql.k8s.enterprisedb.io/v1
kind: Cluster
metadata:
 name: ${cluster_major_upgrade}
spec:
 instances: 1
 imageName: ${postgres_major_upgrade_image}

 storage:
   size: 2Gi

 bootstrap:
   initdb:
     import:
       type: microservice
       databases:
         - app
...
```

©EDB 2025 — ALL RIGHTS RESERVED.

Disaster recovery

Scripts

```
./dr/01-create-all.sh
./dr/02-insert-eu.sh
./dr/03-switchover-cluster.sh
```

Disaster recovery

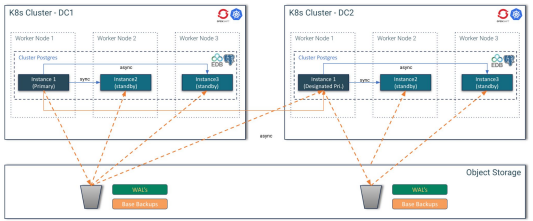
A replica cluster is a CloudNativePG Cluster resource designed to replicate data from another PostgreSQL instance, ideally also managed by CloudNativePG.

Typically, a replica cluster is deployed in a different Kubernetes cluster in another region. These clusters can be configured to perform cascading replication and can rely on object stores for data replication from the source, as detailed further down..

Two clusters:

- pg-eu
- pg-us

Doc link



Code

```
# EU
kubectl delete objectstores.barmancloud.cnpg.io object-storage-eu
envsubst < ../templates/dr/azure-object-storage-eu.yaml >
./tmp/object-storage-eu.yaml
kubectl apply -f ./tmp/object-storage-eu.yaml
sleep 2
rm -f ./tmp/pg-eu.yaml
envsubst < ../templates/dr/pg-eu.yaml > ./tmp/pg-eu.yaml
print_info "Creating pg-eu cluster...\n"
kubectl apply -f ./tmp/pg-eu.yaml
print_info "pg-eu cluster created.\n"

# US
kubectl delete objectstores.barmancloud.cnpg.io object-storage-us
envsubst < ../templates/dr/azure-object-storage-us.yaml >
./tmp/object-storage-us.yaml
kubectl apply -f ./tmp/object-storage-us.yaml
sleep 2
rm -f ./tmp/pg-us.yaml
envsubst < ../templates/dr/pg-us.yaml > ./tmp/pg-us.yaml
kubectl  wait --timeout=30m --for=condition=Ready cluster/pg-eu
print_info "Creating pg-us cluster...\n"
kubectl apply -f ./tmp/pg-us.yaml
./backup_cluster.sh pg-eu
print_info "pg-us cluster created.\n"
```

©EDB 2025 — ALL RIGHTS RESERVED.

The “cnp” plugin for kubectl

- The official CLI for CloudNativePG
  - Available also as RPM or Deb package
- Extends the ‘kubectl’ command:
  - Customize the installation of the operator
  - Status of a cluster
  - Perform a manual switchover (promote a standby) or a restart of a node
  - Issue TLS certificates for client authentication
  - Declare start and stop of a Kubernetes node maintenance
  - Destroy a cluster and all its PVC
  - Fence a cluster or a set of the instances
  - Hibernate a cluster
  - Generate jobs for benchmarking via pgbench and fio
  - Issue a new backup
  - Start pgadmin

©EDB 2025 — ALL RIGHTS RESERVED.

```
Name:               cluster-example
Namespace:          default
System ID:          7100921006673293335
PostgreSQL Image:   ghcr.io/cloudnative-pg/postgresql:14.3
Primary instance:   cluster-example-2
Status:             Cluster in healthy state
Instances:          3
Ready instances:    3
Current Write LSN:  0/C000060 (Timeline: 4 - WAL File: 00000004000000000000000C)

Certificates Status
Certificate Name             Expiration Date                Days Left Until Expiration
----------------             ---------------                --------------------------
cluster-example-replication  2022-08-21 13:15:00 +0000 UTC  89.95
cluster-example-server       2022-08-21 13:15:00 +0000 UTC  89.95
cluster-example-ca           2022-08-21 13:15:00 +0000 UTC  89.95

Continuous Backup status
First Point of Recoverability:  2022-05-23T13:37:08Z
Working WAL archiving:          OK
WALs waiting to be archived:    0
Last Archived WAL:              00000004000000000000000B   @   2022-05-23T13:42:09.37537Z
Last Failed WAL:                -

Streaming Replication status
Name               Sent LSN   Write LSN  Flush LSN  Replay LSN  Write Lag  Flush Lag  Replay Lag  State      Sync State  Sync Priority
----               --------   ---------  ---------  ----------  ---------  ---------  ----------  -----      ----------  -------------
cluster-example-3  0/C000060  0/C000060  0/C000060  0/C000060   00:00:00   00:00:00   00:00:00    streaming  async       0
cluster-example-1  0/C000060  0/C000060  0/C000060  0/C000060   00:00:00   00:00:00   00:00:00    streaming  async       0

Instances status
Name               Database Size  Current LSN  Replication role  Status  QoS         Manager Version
----               -------------  -----------  ----------------  ------  ---         ---------------
cluster-example-3  33 MB          0/C000060    Standby (async)   OK      BestEffort  1.15.0
cluster-example-2  33 MB          0/C000060    Primary           OK      BestEffort  1.15.0
cluster-example-1  33 MB          0/C000060    Standby (async)   OK      BestEffort  1.15.0
```

©EDB 2025— ALL RIGHTS RESERVED.

Install CNPG plugin

NOT NEEDED DURING WORKSHOP
For illustrative purposes.

- In the web terminal run the script 01_install_plugin.sh:

  ./01_install_plugin.sh

©EDB 2025 – ALL RIGHTS RESERVED.

Call to action: Check CNPG plugin

- Call the help for the CNPG Plugin, run:

  kubectl-cnp help

©EDB 2025 — ALL RIGHTS RESERVED.

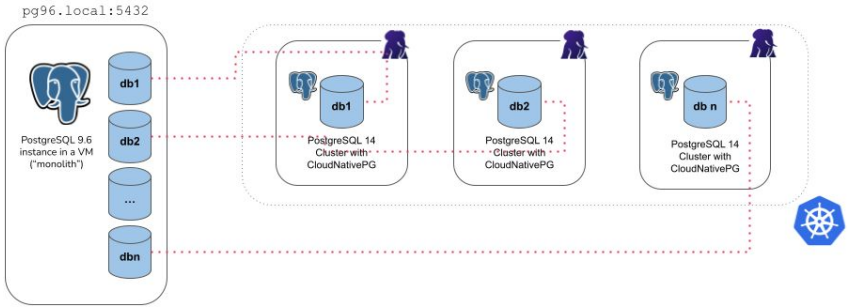
Use case
Database Migration / Major Version Upgrade

©EDB 2025 – ALL RIGHTS RESERVED.

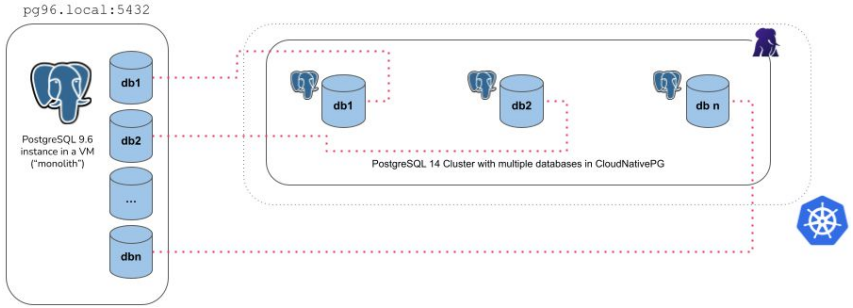
Database Migration

- In this step we will migrate the app database from our existing cluster to the new cluster
- We will create the yaml file with the setting "**import**" in the bootstrap section
- The operator uses internally postgres tools pg_dump and pg_restore
- This method can be used to **migrate** another database or to run **out-of-the-place upgrade**
- Possible settings:

Microservices:



Monolith:



©EDB 2025 – ALL RIGHTS RESERVED.

Call to action: Run migration or out of the place upgrade

- In the web terminal 1:
  - Run the script:

    ./20_upgrade_major_version.sh
  - Check the cluster creation process:

    kubectl get pods -w
  - Check the table test in the cluster-user<X>-17, run the command:
    - Connect to the cluster:

      oc cnp psql cluster-user<X>-17
    - Connect to the database app:

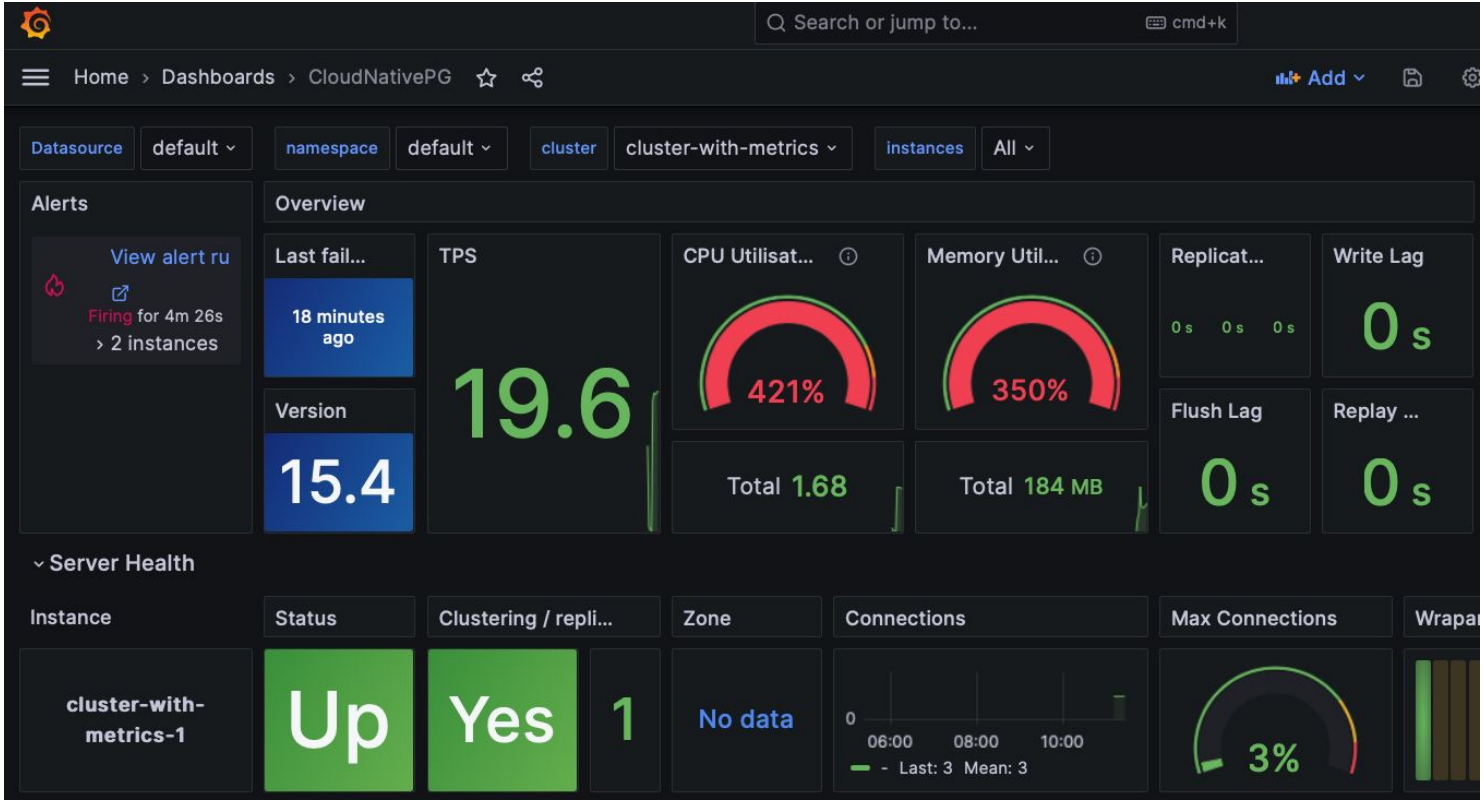
      \c app
    - Run sql commands:

      select version();

      select count(*) from test;

©EDB 2025 – ALL RIGHTS RESERVED.

Grafana Dashboard



©EDB 2025 — ALL RIGHTS RESERVED.